# COM S 3110 Exam 1 Fall 2025

| | |
|---|---|
| **First name** | |
| **Last name** | |
| **Student ID number** | |
| **Lecture section** | |
| **Recitation section** | |

**Some Useful Equalities:** For any positive integer $n$,

- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- $2^{\log_2 n} = n$
- $a^{\log_b n} = n^{\log_b a}, \forall a > 1, b > 1$
- $n^{n/2} \leq n! \leq n^n$
- $\log x^a = a \log x, \forall x > 0$
- $\log(a \times b) = \log a + \log b, \forall a > 0, b > 0$
- $\log(a/b) = \log a - \log b, \forall a > 0, b > 0$
- $\log_b a = \frac{\log_c a}{\log_c b}, \forall a > 0, b > 1, c > 1$
- $a + ar + ar^2 + ... + ar^{n-1} = \frac{a(r^n-1)}{r-1}, \forall r \neq 1$
- $1 + \frac{1}{2} + \frac{1}{2^2} + ... + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
- $1 + 2 + 4 + ... + 2^n = 2^{n+1} - 1$
- $x^{a+b} = x^a \times x^b, \forall x \neq 0$
- $x^{ab} = (x^a)^b, \forall x \neq 0$

**Proof by Contradiction**

1. Assume the negation of the statement you want to prove.
2. Derive the contradiction.
3. This contradiction then negates the assumption.

**Proof by Induction**

1. **Base case:** Prove that the statement is true for the initial value $n = n_0$.
2. **Induction hypothesis:** Assume that the statement holds for some arbitrary integer $n = k \geq n_0$.
3. **Induction step:** Using the inductive hypothesis, prove that the statement also holds for $n = k + 1$.

**Definition (Big-O):** Let $f : \mathbb{N} \to \mathbb{N}$ and $g : \mathbb{N} \to \mathbb{N}$ be functions on the natural numbers. $f(n) \in O(g(n))$ if and only if

$$\exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, f(n) \leq cg(n) \tag{1}$$

1. (10 points) We claim that for all natural numbers $n \geq 4$,

$$n! > 2^n \tag{2}$$

Use mathematical induction to prove this claim by working through the following steps. Recall that for a natural number $n$, the factorial of $n$, denoted $n!$, is defined as

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1. \tag{3}$$

(a) (2 points) Verify the base case.

We need to prove that (2) holds when $n = 4$.

$$\begin{aligned} 4! &= 24 \\ 2^4 &= 16 \end{aligned}$$

So $n! > 2^n$ for $n = 4$.

(b) (2 points) State the induction hypothesis.

Assume that $k! > 2^k$ for an arbitrary $k \geq 4$.

(c) (2 point) State precisely what must be proved in the induction step to complete the induction argument. You do not need to prove it in this part.

We need to prove that $(k+1)! > 2^{k+1}$.

(d) (4 point) Use the induction hypothesis to prove the statement in part (c).

Using the induction hypothesis, $k! > 2^k$, so we get

$$(k+1)! = (k+1)k! > (k+1)2^k$$

Since $k > 1$, we get that $k + 1 > 2$; thus, $(k+1)2^k > 2 \times 2^k = 2^{k+1}$. Combining this with the above inequality, we get $(k+1)! > 2^{k+1}$.

2. (4 points) Consider arbitrary functions $f, g : \mathbb{N} \to \mathbb{N}$. Which of the following conditions are sufficient to conclude that $f(n) \in O(g(n))$?

   **Circle all** that apply. If none apply, you must explicitly write **"None"**. No explanation is needed.

   (a) $f(n) \leq 2g(n)$ for all $n \geq 1$.

   (b) $f(n) \leq cg(n)$ for $n = 3$ and $c = 2$.

   (c) $f(n) \leq cg(n)$ for all $c \geq 1$ if $n = 5$.

   (d) $f(n) \leq cg(n)$ for all $c \geq 1$ if $n \geq 5$.

3. (4 points) Consider arbitrary functions $f, g : \mathbb{N} \to \mathbb{N}$. Suppose you know that

$$f(n) > 5g(n) \text{ for infinitely many } n.$$

   Can you conclude that $f(n) \notin O(g(n))$? Briefly justify your answer.

   We **cannot** conclude that $f(n) \notin O(g(n))$. Even if $f(n) > 5g(n)$ for infinitely many $n$, it can still be that $f(n) \leq cg(n)$ for all sufficiently large $n$.

   For example, consider $f(n) = 10n$ and $g(n) = n$. In this case, $f(n) > 5g(n)$ for all $n \geq 0$. However, $f(n) \leq 10g(n)$ for all $n \geq 0$.

4. (12 points) For each of the following algorithms, **circle all** the valid asymptotic upper bounds (Big-O) for their runtime. No explanation is needed.

(a) (4 points)

```
for (i = 1; i <= n; i++) {
  for (j = i; j <= i+1000; j++) {
    <a constant number of constant time operations>
  }
}
```

$O(n)$ $O(n \log n)$ $O(n^2)$ $O(n^3)$

(b) (4 points)

```
for (i = 1; i <= n; i++) {
  for (j = i; j <= n; j++) {
    <a constant number of constant time operations>
  }
}
```

$O(n)$ $O(n \log n)$ $O(n^2)$ $O(n^3)$

(c) (4 points)

```
for (i = 1; i <= n; i++) {
  for (j = 1; j <= n*i; j++) {
    <a constant number of constant time operations>
  }
}
```

$O(n)$ $O(n \log n)$ $O(n^2)$ $O(n^3)$

5. (20 points) Consider the following algorithms:

```
i = n
while i >= 8 {
  for (j = 1; j<=i; j++) {
    <perform a constant time primitive operation>
  }
  i = i / 2
}
```

(a) (4 points) Complete the table below, expressing each entry **only in terms of** $n$, $k$ and constants.

| Outer loop iteration | Value of $i$ at the beginning of iteration | Number of iterations of the inner loop |
|---|---|---|
| 1 | $n$ | $n$ |
| 2 | $\frac{n}{2}$ | $\frac{n}{2}$ |
| 3 | $\frac{n}{2^2}$ | $\frac{n}{2^2}$ |
| 4 | $\frac{n}{2^3}$ | $\frac{n}{2^3}$ |
| $\vdots$ | $\vdots$ | $\vdots$ |
| $k$ | $\frac{n}{2^{k-1}}$ | $\frac{n}{2^{k-1}}$ |

(b) (3 points) Assume that $n$ is a power of 2, i.e., $\log_2 n$ is an integer. Determine the **exact number of iterations** that the `while` loop executes before terminating. Your answer must be in terms of $n$, **including all constant factors**. Do not give an asymptotic (Big-O) bound.

At the $k^{th}$ iteration of the outer loop, $i = \frac{n}{2^{k-1}}$. According to the loop condition, we get

$$\begin{aligned} \frac{n}{2^{k-1}} &\geq 8 \\ 2^{k+2} &\leq n \\ k &\leq \log n - 2 \end{aligned}$$

So the while loop can execute at most $\log n - 2$ times. Since $\log n$ is an integer, in the last iteration, we get the equality $k = \log n - 2$, meaning that the while loop runs exactly $\log n - 2$ times.

(c) (3 points) Let $K$ be the exact number of iterations of the `while` loop (as determined in part (b)). Determine the **exact** total number of times the body of the `for` loop executes, i.e., the total number of executions of the line `<perform a constant time primitive operation>` over the entire run of the algorithm. Your answer must be expressed as a function of $n$ and $K$, including **all constant factors**. If your reasoning leads to a summation, you must **explicitly evaluate** the summation. You can leave $K$ as a variable and do not need to substitute the result from part (b). Do not give an asymptotic Big-O bound.

$$\begin{aligned} \sum_{i=1}^{K} \frac{n}{2^{i-1}} &= n \sum_{i=1}^{K} \frac{1}{2^{i-1}} \\ &= 2n(1 - \frac{1}{2^K}) \end{aligned}$$

(d) (10 points) Assume that $n$ is a power of 2. Define the following constants to represent the execution time of each operations:

- $c_1$: time required to perform the initial assignment `i = n`.
- $c_2$: time required to evaluate `i >= 8`.
- $c_3$: time required to update `i = i / 2`.
- $c_4$: time required to set up the `for` loop before any iterations begin, e.g., initializing `j`.
- $c_5$: time required to evaluate `j <= i`.
- $c_6$: time required to increment `j++`.
- $c_7$: time required to perform the primitive operation inside the body of the `for` loop.

Determine the **exact** total runtime of the algorithm in terms of $n, c_1, c_2, \ldots, c_7$, **including all constant factors**, and making sure to account for **every single line of the pseudocode**. After obtaining the exact runtime, simplify it and determine the corresponding Big-O upper bound.

Note: Giving only the Big-O bound without first deriving the exact runtime will receive zero credit.

The line $i = n$ is executed only once, so the total time is $c_1$.

From part (b), the `while` loop runs exactly $\log n - 2$ times. As a result, the evaluation of $i \geq 8$ runs $\log n - 1$ times (the last one is when $i < 8$ and the loop terminates). On the other hand, the update $i = i/2$ runs $\log n - 2$ times. The total time required for these two lines is, therefore,

$$c_2(\log n - 1) + c_3(\log n - 2)$$

The `for` loop is setup only once for each iteration of the `while` loop. So the total time for setting up the `for` loop is

$$c_4(\log n - 2)$$

From part (c), the `for` loop runs exactly $2n(1 - \frac{1}{2^K})$ times, with $K = \log n - 2$. Each iteration of the `for` loop takes time $c_5 + c_6 + c_7$. An additional evaluation `j <= i` is also required before the loop terminates (when `j > i`). Thus, the total time of the `for` loop (excluding the setup time) is

$$\begin{aligned} 2n(1 - \frac{1}{2^{\log n - 2}})(c_5 + c_6 + c_7) + c_5(\log n - 2) &= 2n(1 - \frac{4}{n})(c_5 + c_6 + c_7) + c_5(\log n - 2) \\ &= 2(c_5 + c_6 + c_7)(n - 4) + c_5 \log n - 2c_5 \end{aligned}$$

Summing up everything, we get that the total runtime of the algorithm is

$$\begin{aligned} & c_1 + c_2(\log n - 1) + c_3(\log n - 2) + c_4(\log n - 2) + 2(c_5 + c_6 + c_7)(n - 4) + c_5 \log n - 2c_5 \\ = \; & (c_2 + c_3 + c_4 + c_5)(\log n) + 2(c_5 + c_6 + c_7)n + (c_1 - c_2 - 2c_3 - 2c_4 - 10c_5 - 8c_6 - 8c_7) \\ = \; & c_1' \log n + c_2' n + c_3' \\ \in \; & O(n) \end{aligned}$$

where $c_1' = c_2 + c_3 + c_4 + c_5$, $c_2' = 2(c_5 + c_6 + c_7)$, and $c_3' = c_1 - c_2 - 2c_3 - 2c_4 - 10c_5 - 8c_6 - 8c_7$.

| **Question** | **Points** | **Score** |
|---|---|---|
| 1 | 10 | |
| 2 | 4 | |
| 3 | 4 | |
| 4 | 12 | |
| 5(a) | 4 | |
| 5(b) | 3 | |
| 5(c) | 3 | |
| 5(d) | 10 | |
| Total: | 50 | |